AI & ML Practical

Foundation of Artificial Intelligence and Machine Learning

Lab Course CS-321-VSC-P | T.Y.B.Sc.(C.S.) Sem V | SPPU

Solved Slips

Slip 1, 2, 3, 4, 7, 8, 9, 14, 15, 16, 17, 21, 22, 23, 25

Every slip: Q1, Q2 and its OR option, each with the question, the program and its output.

SLIP NO. 1

AI & ML Practical | 35 Marks

(1) Write a program to implement Breadth First Search (BFS) for the Monkey Banana Problem. [10 marks]

PROGRAM

```
from collections import deque

LOC = ['door', 'window', 'center']
BANANA = 'center'

def moves(state):
    m, on, b, has = state
    res = []
    if not on:
        for l in LOC:
            if l != m:
                res.append(((l, False, b, has), 'Walk ' + m + ' -> ' + l))
        if m == b:
            for l in LOC:
                if l != b:
                    res.append(((l, False, l, has), 'Push box ' + b + ' -> ' + l))
            res.append(((m, True, b, has), 'Climb on box'))
    else:
        res.append(((m, False, b, has), 'Climb down'))
        if m == BANANA:
            res.append(((m, True, b, True), 'Grab banana'))
    return res

def bfs(start):
    queue = deque([(start, [])])
    seen = {start}
    while queue:
        state, path = queue.popleft()
        if state[3]:
            return path
        for nxt, action in moves(state):
            if nxt not in seen:
                seen.add(nxt)
                queue.append((nxt, path + [action]))

start = ('door', False, 'window', False)
for i, step in enumerate(bfs(start), 1):
    print(i, step)
print('Goal: Banana obtained!')
```

OUTPUT

```
1 Walk door -> window
2 Push box window -> center
3 Climb on box
4 Grab banana
Goal: Banana obtained!
```

(2) Write a program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach. [20 marks]

PROGRAM

```
def map_coloring(graph, colors):
    assign = {}
```

```
    vertices = list(graph)

    def is_safe(v, c):
        return all(assign.get(n) != c for n in graph[v])

    def solve(i):
        if i == len(vertices):
            return True
        v = vertices[i]
        for c in colors:
            if is_safe(v, c):
                assign[v] = c
                if solve(i + 1):
                    return True
                del assign[v]
        return False

    return assign if solve(0) else None

graph = {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'D'], 'D': ['B', 'C']}
colors = ['Red', 'Green', 'Blue']
result = map_coloring(graph, colors)
if result:
    print('Map Coloring Solution:')
    for v, c in result.items():
        print(v, '->', c)
else:
    print('No solution with', len(colors), 'colors')
```

OUTPUT

```
Map Coloring Solution:
A -> Red
B -> Green
C -> Green
D -> Red
```

— OR —

(2) **Write a program to implement the A* Search Algorithm to find the shortest path between a source node and a destination node. [20 marks]**

PROGRAM

```
import heapq

graph = {'A': {'B': 1, 'C': 4}, 'B': {'D': 2, 'E': 5}, 'C': {'F': 3},
         'D': {'G': 3}, 'E': {'G': 1}, 'F': {'G': 2}, 'G': {}}
h = {'A': 6, 'B': 5, 'C': 4, 'D': 3, 'E': 2, 'F': 2, 'G': 0}

def astar(graph, h, start, goal):
    pq = [(h[start], 0, start, [start])]
    best = {}
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g
        if node in best and best[node] <= g:
            continue
        best[node] = g
        for nb, cost in graph[node].items():
            heapq.heappush(pq, (g + cost + h[nb], g + cost, nb, path + [nb]))
    return None, float('inf')

path, cost = astar(graph, h, 'A', 'G')
```

```
print('Shortest path:', ' -> '.join(path))
print('Total cost:', cost)
```

OUTPUT

```
Shortest path: A -> B -> D -> G
Total cost: 6
```

SLIP NO. 2

AI & ML Practical | 35 Marks

(1) Write a program to solve the Water Jug Problem using BFS. [10 marks]

PROGRAM

```
from collections import deque

def water_jug(cap1, cap2, goal):
    start = (0, 0)
    queue = deque([(start, [start])])
    seen = {start}
    while queue:
        (a, b), path = queue.popleft()
        if a == goal or b == goal:
            return path
        p12 = min(a, cap2 - b)
        p21 = min(b, cap1 - a)
        for nxt in [(cap1, b), (a, cap2), (0, b), (a, 0),
                    (a - p12, b + p12), (a + p21, b - p21)]:
            if nxt not in seen:
                seen.add(nxt)
                queue.append((nxt, path + [nxt]))
    return None

solution = water_jug(4, 3, 2)
print('Steps (4L jug, 3L jug):')
for s in solution:
    print(s)
```

OUTPUT

```
Steps (4L jug, 3L jug):
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
```

(2) Write a program to implement the Backtracking Algorithm for solving a constraint satisfaction problem. [20 marks]

PROGRAM

```
variables = [1, 2, 3, 4, 5]
domain = ['Red', 'Green', 'Blue', 'Yellow', 'Purple']

neighbors = {v: [u for u in variables if u != v] for v in variables}

def consistent(var, val, assignment):
    return all(assignment.get(n) != val for n in neighbors[var])

def backtrack(assignment):
    if len(assignment) == len(variables):
        return assignment
    var = [v for v in variables if v not in assignment][0]
    for val in domain:
        if consistent(var, val, assignment):
            assignment[var] = val
            result = backtrack(assignment)
            if result:
                return result
```

```
            del assignment[var]
    return None

print(backtrack({}))
```

OUTPUT

```
{1: 'Red', 2: 'Green', 3: 'Blue', 4: 'Yellow', 5: 'Purple'}
```

— OR —

(2) **Write a program to implement a Game Tree Representation for a two-player game. [20 marks]**

PROGRAM

```
class Node:
    def __init__(self, name, value=None):
        self.name = name
        self.value = value
        self.children = []

    def add(self, *kids):
        self.children.extend(kids)
        return self

def show(node, depth=0):
    label = node.name + (' [' + str(node.value) + ']' if node.value is not None else '')
    print('    ' * depth + label)
    for child in node.children:
        show(child, depth + 1)

def minimax(node, maximizing):
    if not node.children:
        return node.value
    values = [minimax(c, not maximizing) for c in node.children]
    return max(values) if maximizing else min(values)

root = Node('A (MAX)').add(
    Node('B (MIN)').add(Node('D', 3), Node('E', 5)),
    Node('C (MIN)').add(Node('F', 2), Node('G', 9)))

print('Game Tree:')
show(root)
print('Value of root:', minimax(root, True))
```

OUTPUT

```
Game Tree:
A (MAX)
    B (MIN)
        D [3]
        E [5]
    C (MIN)
        F [2]
        G [9]
Value of root: 3
```

SLIP NO. 3

AI & ML Practical | 35 Marks

(1) Write a program to implement BFS traversal for a graph. [10 marks]

PROGRAM

```
from collections import deque

def bfs(graph, start):
    visited = [start]
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for nb in graph[node]:
            if nb not in visited:
                visited.append(nb)
                queue.append(nb)
    return visited

graph = {i: [2 * i, 2 * i + 1] for i in range(1, 8)}
for leaf in range(8, 16):
    graph[leaf] = []
print('BFS traversal:', bfs(graph, 1))
```

OUTPUT

```
BFS traversal: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

(2) Write a program to implement Alpha-Beta Pruning for optimizing the Minimax search process. [20 marks]

PROGRAM

```
import math

def alphabeta(node, alpha, beta, maximizing):
    if isinstance(node, int):
        return node
    if maximizing:
        v = -math.inf
        for child in node:
            v = max(v, alphabeta(child, alpha, beta, False))
            alpha = max(alpha, v)
            if alpha >= beta:
                print('Pruned (beta cut-off)')
                break
        return v
    v = math.inf
    for child in node:
        v = min(v, alphabeta(child, alpha, beta, True))
        beta = min(beta, v)
        if alpha >= beta:
            print('Pruned remaining children (alpha cut-off)')
            break
    return v

tree = [[3, 5], [2, 9], [0, 7]]
print('Best value for MAX:', alphabeta(tree, -math.inf, math.inf, True))
```

OUTPUT

```
Pruned remaining children (alpha cut-off)
Pruned remaining children (alpha cut-off)
```

```
Best value for MAX: 3
```

— OR —

(2) **Write a program to implement Propositional Logic and evaluate logical expressions using operators such as AND, OR, and NOT. [20 marks]**

PROGRAM

```python
import itertools

AND = lambda p, q: p and q
OR = lambda p, q: p or q
NOT = lambda p: not p
IMPLIES = lambda p, q: (not p) or q

def evaluate(expr, **values):
    env = {'AND': AND, 'OR': OR, 'NOT': NOT, 'IMPLIES': IMPLIES}
    env.update(values)
    return eval(expr, {'__builtins__': {}}, env)

def truth_table(expr, names):
    print(*names, '|', expr)
    for vals in itertools.product([True, False], repeat=len(names)):
        r = evaluate(expr, **dict(zip(names, vals)))
        print(*[int(v) for v in vals], '|', int(r))

print(evaluate('AND(p, NOT(q))', p=True, q=False))
print(evaluate('OR(p, q)', p=False, q=False))
print(evaluate('NOT(p)', p=True))
print()
truth_table('OR(AND(p, q), NOT(r))', ['p', 'q', 'r'])
```

OUTPUT

```
True
False
False

p q r | OR(AND(p, q), NOT(r))
1 1 1 | 1
1 1 0 | 1
1 0 1 | 0
1 0 0 | 1
0 1 1 | 0
0 1 0 | 1
0 0 1 | 0
0 0 0 | 1
```

SLIP NO. 4

AI & ML Practical | 35 Marks

(1) **Write a program to solve a state-space search problem using BFS. Start Vertex = A. [10 marks]**

PROGRAM

```
from collections import deque

def bfs(graph, start):
    visited = [start]
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for nb in graph[node]:
            if nb not in visited:
                visited.append(nb)
                queue.append(nb)
    return visited

graph = {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'E'],
         'D': ['B', 'E'], 'E': ['C', 'D']}
print('BFS traversal:', bfs(graph, 'A'))
```

OUTPUT

```
BFS traversal: ['A', 'B', 'C', 'D', 'E']
```

(2) **Write a program to implement Predicate Logic using predicates, variables, and quantifier. [20 marks]**

PROGRAM

```
people = ['Ram', 'Sita', 'Hari', 'Gita']
human = {'Ram', 'Sita', 'Hari', 'Gita'}
mortal = set(human)
parent = {('Ram', 'Hari'), ('Sita', 'Hari'), ('Hari', 'Gita')}

def Human(x): return x in human
def Mortal(x): return x in mortal
def Parent(x, y): return (x, y) in parent

def forall(pred, domain): return all(pred(x) for x in domain)
def exists(pred, domain): return any(pred(x) for x in domain)

print('forall x: Human(x) -> Mortal(x) :', forall(lambda x: (not Human(x)) or Mortal(x), people))
print('exists x: Parent(x, Hari)        :', exists(lambda x: Parent(x, 'Hari'), people))
print('Grandparent(Ram, Gita)           :', exists(lambda y: Parent('Ram', y) and Parent(y, 'Gita'), people))
print('forall x: Parent(x, Hari)        :', forall(lambda x: Parent(x, 'Hari'), people))
```

OUTPUT

```
forall x: Human(x) -> Mortal(x) : True
exists x: Parent(x, Hari)       : True
Grandparent(Ram, Gita)          : True
forall x: Parent(x, Hari)       : False
```

— OR —

(2) **Write a program to implement Gaussian NB classifier with 3 classes on numeric data, use make_blobs to create a dataset X of 200 sample points in 2D Space, 3 clusters and each cluster spread (SD) over 1.5. For testing (Prediction), use the following values to test the Gaussian NB Classifier. Input (Values): [-2, 5], [0,0], [6, -0.3] Output: [0 1 1]. Print the accuracy score using the accuracy_score() from sklearn.metrics library. [20 marks]**

Note: random_state=42 gives the expected output [0 1 1].

PROGRAM

```
from sklearn.datasets import make_blobs
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

X, y = make_blobs(n_samples=200, centers=3, n_features=2,
                  cluster_std=1.5, random_state=42)
model = GaussianNB()
model.fit(X, y)

X_test = [[-2, 5], [0, 0], [6, -0.3]]
pred = model.predict(X_test)
print('Prediction:', pred)
print('Accuracy:', accuracy_score([0, 1, 1], pred))
```

OUTPUT

```
Prediction: [0 1 1]
Accuracy: 1.0
```

SLIP NO. 7

AI & ML Practical | 35 Marks

(1) **Write a program to implement the A* Search Algorithm to find the shortest path between a source node and a destination node. Source = V1, Destination = V6. [10 marks]**

Note: heuristic values are not given in the paper, so admissible values are assumed (V1=6, V2=4, V3=3, V4=2, V5=1, V6=0).

PROGRAM

```
import heapq

graph = {'V1': {'V2': 2, 'V3': 4}, 'V2': {'V1': 2, 'V4': 3},
         'V3': {'V1': 4, 'V4': 1, 'V5': 5}, 'V4': {'V2': 3, 'V3': 1, 'V6': 2},
         'V5': {'V3': 5, 'V6': 1}, 'V6': {'V4': 2, 'V5': 1}}
h = {'V1': 6, 'V2': 4, 'V3': 3, 'V4': 2, 'V5': 1, 'V6': 0}

def astar(graph, h, start, goal):
    pq = [(h[start], 0, start, [start])]
    best = {}
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g
        if node in best and best[node] <= g:
            continue
        best[node] = g
        for nb, cost in graph[node].items():
            heapq.heappush(pq, (g + cost + h[nb], g + cost, nb, path + [nb]))
    return None, float('inf')

path, cost = astar(graph, h, 'V1', 'V6')
print('Shortest path:', ' -> '.join(path))
print('Total cost:', cost)
```

OUTPUT

```
Shortest path: V1 -> V2 -> V4 -> V6
Total cost: 7
```

(2) **Write a program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach. [20 marks]**

PROGRAM

```
def map_coloring(graph, colors):
    assign = {}
    vertices = list(graph)

    def is_safe(v, c):
        return all(assign.get(n) != c for n in graph[v])

    def solve(i):
        if i == len(vertices):
            return True
        v = vertices[i]
        for c in colors:
            if is_safe(v, c):
                assign[v] = c
                if solve(i + 1):
                    return True
```

```
                del assign[v]
        return False

    return assign if solve(0) else None

graph = {'N1': ['N2', 'N3'], 'N2': ['N1', 'N4'], 'N3': ['N1', 'N5'],
         'N4': ['N2', 'N5'], 'N5': ['N3', 'N4']}
colors = ['Red', 'Green', 'Blue']
result = map_coloring(graph, colors)
if result:
    print('Map Coloring Solution:')
    for v, c in result.items():
        print(v, '->', c)
else:
    print('No solution with', len(colors), 'colors')
```

OUTPUT

```
Map Coloring Solution:
N1 -> Red
N2 -> Green
N3 -> Green
N4 -> Red
N5 -> Blue
```

— OR —

(2) **Write a program to develop a Tic-Tac-Toe Game in which the computer uses the Minimax Algorithm to play optimally. [20 marks]**

PROGRAM

```
import math

WINS = [(0,1,2), (3,4,5), (6,7,8), (0,3,6), (1,4,7), (2,5,8), (0,4,8), (2,4,6)]

def winner(b):
    for x, y, z in WINS:
        if b[x] != ' ' and b[x] == b[y] == b[z]:
            return b[x]
    return None

def minimax(b, is_max):
    w = winner(b)
    if w == 'O': return 1
    if w == 'X': return -1
    if ' ' not in b: return 0
    scores = []
    for i in range(9):
        if b[i] == ' ':
            b[i] = 'O' if is_max else 'X'
            scores.append(minimax(b, not is_max))
            b[i] = ' '
    return max(scores) if is_max else min(scores)

def best_move(b):
    best, move = -math.inf, None
    for i in range(9):
        if b[i] == ' ':
            b[i] = 'O'
            s = minimax(b, False)
            b[i] = ' '
            if s > best:
                best, move = s, i
    return move
```

```
def show(b):
    for i in range(0, 9, 3):
        print(' | '.join(b[i:i + 3]))
    print()

def play():
    b = [' '] * 9
    for turn in range(9):
        if turn % 2 == 0:
            m = int(input('Your move X (0-8): '))
            while m not in range(9) or b[m] != ' ':
                m = int(input('Invalid, try again: '))
            b[m] = 'X'
        else:
            b[best_move(b)] = 'O'
        show(b)
        w = winner(b)
        if w:
            print(w, 'wins!')
            return
    print('Draw')

play()
```

OUTPUT

```
Your move X (0-8): 4
  |   |
  | X |
  |   |

O |   |
  | X |
  |   |

Your move X (0-8): 0
Invalid, try again: 1
O | X |
  | X |
  |   |

O | X |
  | X |
  | O |

Your move X (0-8): 2
O | X | X
  | X |
  | O |

O | X | X
  | X |
O | O |

Your move X (0-8): 3
O | X | X
X | X |
O | O |

O | X | X
X | X |
O | O | O

O wins!
```

SLIP NO. 8

AI & ML Practical | 35 Marks

(1) **Write a program to solve the 4-Puzzle Problem using the A* Search Algorithm. [10 marks]**

PROGRAM

```
import heapq

GOAL = (1, 2, 3, 0)
NEI = {0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2]}
GOAL_POS = {1: (0, 0), 2: (0, 1), 3: (1, 0)}

def h(s):
    d = 0
    for i, t in enumerate(s):
        if t:
            gr, gc = GOAL_POS[t]
            d += abs(i // 2 - gr) + abs(i % 2 - gc)
    return d

def solve(start):
    pq = [(h(start), 0, start, [start])]
    seen = {}
    while pq:
        f, g, s, path = heapq.heappop(pq)
        if s == GOAL:
            return path
        if s in seen and seen[s] <= g:
            continue
        seen[s] = g
        b = s.index(0)
        for n in NEI[b]:
            l = list(s)
            l[b], l[n] = l[n], l[b]
            t = tuple(l)
            heapq.heappush(pq, (g + 1 + h(t), g + 1, t, path + [t]))
    return None

start = (0, 1, 3, 2)
sol = solve(start)
for i, s in enumerate(sol):
    print('Step', i)
    print(s[0], s[1])
    print(s[2], s[3])
print('Moves:', len(sol) - 1)
```

OUTPUT

```
Step 0
0 1
3 2
Step 1
1 0
3 2
Step 2
1 2
3 0
Moves: 2
```

(2) **Write a program or report that compares Minimax and Alpha-Beta Pruning on the same game tree. [20 marks]**

PROGRAM

```
import math

tree = [[3, 5], [2, 9], [0, 7]]
count = {'minimax': 0, 'alphabeta': 0}

def minimax(node, maxp):
    if isinstance(node, int):
        count['minimax'] += 1
        return node
    vals = [minimax(c, not maxp) for c in node]
    return max(vals) if maxp else min(vals)

def alphabeta(node, alpha, beta, maxp):
    if isinstance(node, int):
        count['alphabeta'] += 1
        return node
    if maxp:
        v = -math.inf
        for c in node:
            v = max(v, alphabeta(c, alpha, beta, False))
            alpha = max(alpha, v)
            if alpha >= beta:
                break
        return v
    v = math.inf
    for c in node:
        v = min(v, alphabeta(c, alpha, beta, True))
        beta = min(beta, v)
        if alpha >= beta:
            break
    return v

print('Minimax    value:', minimax(tree, True), '| leaf nodes evaluated:', count['minimax'])
print('Alpha-Beta value:', alphabeta(tree, -math.inf, math.inf, True), '| leaf nodes evaluated:', count['alphabeta'])
```

OUTPUT

```
Minimax    value: 3 | leaf nodes evaluated: 6
Alpha-Beta value: 3 | leaf nodes evaluated: 4
```

— OR —

(2) **Write a program to design a Rule-Based Expert System for simple decision-making. [20 marks]**

PROGRAM

```
rules = [
    ({'fever', 'cough', 'breathless'}, 'Possible chest infection - see a doctor immediately'),
    ({'fever', 'cough'}, 'Probably flu - take rest and fluids'),
    ({'sneezing', 'runny_nose'}, 'Common cold - warm drinks and rest'),
    ({'fever', 'rash'}, 'Possible viral rash - consult a doctor'),
    ({'headache'}, 'Mild headache - rest and drink water'),
]

print('Symptoms: fever, cough, breathless, sneezing, runny_nose, rash, headache')
facts = set(input('Enter your symptoms (comma separated): ').replace(' ', '').split(','))

for cond, advice in rules:
    if cond <= facts:
        print('Diagnosis:', advice)
        break
else:
    print('No rule matched - please consult a doctor')
```

OUTPUT

```
Symptoms: fever, cough, breathless, sneezing, runny_nose, rash, headache
Enter your symptoms (comma separated): fever,cough
Diagnosis: Probably flu - take rest and fluids
```

SLIP NO. 9

AI & ML Practical | 35 Marks

(1) **Write a program to demonstrate the Hill Climbing Algorithm for finding the maximum value of a simple objective function. [10 marks]**

PROGRAM

```
import random

def f(x):
    return -x ** 2 + 4 * x

def hill_climb(f, start, step=0.5, iters=1000):
    x = start
    for _ in range(iters):
        best = max([x + step, x - step], key=f)
        if f(best) <= f(x):
            break
        x = best
    return x, f(x)

random.seed(1)
start = random.randint(-10, 10)
x, val = hill_climb(f, start)
print('Starting point:', start)
print('Maximum found at x =', round(x, 2))
print('Maximum value F(x) =', round(val, 2))
```

OUTPUT

```
Starting point: -6
Maximum found at x = 2.0
Maximum value F(x) = 4.0
```

(2) **Write a program to implement the Forward Chaining inference mechanism. [20 marks]**

PROGRAM

```
rules = [({'croaks', 'eats_flies'}, 'frog'),
         ({'chirps', 'sings'}, 'canary'),
         ({'frog'}, 'green'),
         ({'canary'}, 'yellow')]
facts = {'croaks', 'eats_flies'}
goal = 'green'

def forward_chain(rules, facts, goal):
    facts = set(facts)
    changed = True
    while changed:
        changed = False
        for cond, result in rules:
            if cond <= facts and result not in facts:
                facts.add(result)
                print(sorted(cond), '=>', result)
                changed = True
                if result == goal:
                    return True
    return goal in facts

print('Goal', goal, 'proved:', forward_chain(rules, facts, goal))
```

OUTPUT

```
['croaks', 'eats_flies'] => frog
['frog'] => green
Goal green proved: True
```

— OR —

(2) **Write a program to implement the Decision Tree Classifier for a classification problem. Use the dataset provided with columns: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age, Outcome (class label 0 or 1). Split the dataset into 70:30 train:test. Evaluate the model using accuracy_score(). Optimize the model's performance by tuning the criterion (default is GINI, use entropy) and max_depth (=2, 3 or 4) parameter of the DT Classifier. [20 marks]**

PROGRAM

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

data = [[6,148,72,35,0,33.6,0.627,50,1], [1,85,66,29,0,26.6,0.351,31,0],
        [8,183,64,0,0,23.3,0.672,32,1], [1,89,66,23,94,28.1,0.167,21,0],
        [0,137,40,35,168,43.1,2.288,33,1], [5,116,74,0,0,25.6,0.201,30,0],
        [3,78,50,32,88,31,0.248,26,1], [10,115,0,0,0,35.3,0.134,29,0],
        [2,197,70,45,543,30.5,0.158,53,1], [8,125,96,0,0,0,0.232,54,1],
        [4,110,92,0,0,37.6,0.191,30,0]]
cols = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
        'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome']
df = pd.DataFrame(data, columns=cols)

X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=7)

best = (0, None)
for crit in ['gini', 'entropy']:
    for d in [2, 3, 4]:
        model = DecisionTreeClassifier(criterion=crit, max_depth=d, random_state=1)
        model.fit(X_train, y_train)
        acc = accuracy_score(y_test, model.predict(X_test))
        print('criterion =', crit, '| max_depth =', d, '| accuracy =', round(acc, 2))
        if acc > best[0]:
            best = (acc, (crit, d))
print('Best accuracy', round(best[0], 2), 'with (criterion, max_depth) =', best[1])
```

OUTPUT

```
criterion = gini | max_depth = 2 | accuracy = 0.5
criterion = gini | max_depth = 3 | accuracy = 0.75
criterion = gini | max_depth = 4 | accuracy = 0.75
criterion = entropy | max_depth = 2 | accuracy = 0.5
criterion = entropy | max_depth = 3 | accuracy = 0.75
criterion = entropy | max_depth = 4 | accuracy = 0.75
Best accuracy 0.75 with (criterion, max_depth) = ('gini', 3)
```

SLIP NO. 14

AI & ML Practical | 35 Marks

(1) Write a program to implement BFS traversal for a graph. [10 marks]

PROGRAM

```
from collections import deque

def bfs(graph, start):
    visited = [start]
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for nb in graph[node]:
            if nb not in visited:
                visited.append(nb)
                queue.append(nb)
    return visited

graph = {'V1': ['V2', 'V3', 'V5'], 'V2': ['V1', 'V4'], 'V3': ['V1', 'V6'],
         'V4': ['V2', 'V5'], 'V5': ['V1', 'V4', 'V6'], 'V6': ['V3', 'V5']}
print('BFS traversal:', bfs(graph, 'V1'))
```

OUTPUT

```
BFS traversal: ['V1', 'V2', 'V3', 'V5', 'V4', 'V6']
```

(2) Write a program or report to compare Supervised Learning and Unsupervised Learning techniques. [20 marks]

PROGRAM

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score, silhouette_score, adjusted_rand_score

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
knn = KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)
print('Supervised (KNN) accuracy:', round(accuracy_score(y_test, knn.predict(X_test)), 3))

km = KMeans(n_clusters=3, n_init=10, random_state=1).fit(X)
print('Unsupervised (KMeans) silhouette:', round(silhouette_score(X, km.labels_), 3))
print('Agreement with true labels (ARI):', round(adjusted_rand_score(y, km.labels_), 3))

print()
print('Supervised   : labelled data, predicts a label/value (KNN, SVM, Decision Tree)')
print('Unsupervised : unlabelled data, finds groups/structure (K-Means, PCA)')
```

OUTPUT

```
Supervised (KNN) accuracy: 0.978
Unsupervised (KMeans) silhouette: 0.553
Agreement with true labels (ARI): 0.73

Supervised   : labelled data, predicts a label/value (KNN, SVM, Decision Tree)
Unsupervised : unlabelled data, finds groups/structure (K-Means, PCA)
```

— OR —

(2) Write a program to demonstrate basic Ontological Engineering concepts using classes, subclasses and relationships. [20 marks]

PROGRAM

```
class Entity:
    def __init__(self, name):
        self.name = name
        self.relations = {}

    def relate(self, rel, other):
        self.relations.setdefault(rel, []).append(other)

    def __repr__(self):
        return self.name

class Animal(Entity): legs = 4
class Mammal(Animal): feeds_milk = True
class Bird(Animal):
    legs = 2
    can_fly = True
class Dog(Mammal): sound = 'Bark'
class Eagle(Bird): pass
class Person(Entity): pass

def hierarchy(cls, level=0):
    print('  ' * level + cls.__name__)
    for sub in cls.__subclasses__():
        hierarchy(sub, level + 1)

hierarchy(Entity)

tommy = Dog('Tommy')
ram = Person('Ram')
ram.relate('owns', tommy)

print('Tommy is a Mammal?', isinstance(tommy, Mammal))
print('Tommy is an Animal?', isinstance(tommy, Animal))
print('Tommy is a Bird?', isinstance(tommy, Bird))
print('is-a chain:', [c.__name__ for c in Dog.__mro__ if c is not object])
print(ram, 'owns', ram.relations['owns'])
print('Sound:', tommy.sound, '| Feeds milk:', tommy.feeds_milk, '| Legs:', tommy.legs)
```

OUTPUT

```
Entity
  Animal
    Mammal
      Dog
    Bird
      Eagle
  Person
Tommy is a Mammal? True
Tommy is an Animal? True
Tommy is a Bird? False
is-a chain: ['Dog', 'Mammal', 'Animal', 'Entity']
Ram owns [Tommy]
Sound: Bark | Feeds milk: True | Legs: 4
```

SLIP NO. 15

AI & ML Practical | 35 Marks

(1) **Write a program to solve a state-space search problem using DFS. Start vertex - 1. [10 marks]**

PROGRAM

```
def dfs(graph, node, visited=None):
    if visited is None:
        visited = []
    visited.append(node)
    for nb in graph[node]:
        if nb not in visited:
            dfs(graph, nb, visited)
    return visited

graph = {1: [2, 3], 2: [1, 4], 3: [1, 5], 4: [2, 5, 6], 5: [3, 4, 6], 6: [4, 5]}
print('DFS traversal from 1:', dfs(graph, 1))
```

OUTPUT

```
DFS traversal from 1: [1, 2, 4, 5, 3, 6]
```

(2) **Write Python program to demonstrate the following objective functions used in clustering algorithms (any 2): K-Means Inertia (WCSS), Gaussian Mixture Model (GMM) Log-Likelihood, Silhouette Coefficient, Davies-Bouldin Index. Using matplotlib, plot the Elbow curve, Silhouette scores, and Davies-Bouldin scores across different values of K (on x-axis). [20 marks]**

PROGRAM

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=1.0, random_state=42)
ks = list(range(2, 11))
wcss, sil, db = [], [], []
for k in ks:
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    wcss.append(km.inertia_)
    sil.append(silhouette_score(X, km.labels_))
    db.append(davies_bouldin_score(X, km.labels_))
    print('K =', k, '| WCSS =', round(wcss[-1], 1), '| Silhouette =', round(sil[-1], 3), '| DB =', round(db[-1], 3))

fig, ax = plt.subplots(1, 3, figsize=(15, 4))
ax[0].plot(ks, wcss, 'o-'); ax[0].set_title('Elbow curve (WCSS / Inertia)')
ax[1].plot(ks, sil, 'o-'); ax[1].set_title('Silhouette score (higher is better)')
ax[2].plot(ks, db, 'o-'); ax[2].set_title('Davies-Bouldin (lower is better)')
for a in ax:
    a.set_xlabel('Number of clusters K')
plt.tight_layout()
plt.show()
print('Best K by silhouette:', ks[sil.index(max(sil))])
```

OUTPUT

```
K = 2 | WCSS = 9416.2 | Silhouette = 0.59 | DB = 0.525
K = 3 | WCSS = 2110.4 | Silhouette = 0.757 | DB = 0.356
K = 4 | WCSS = 564.9 | Silhouette = 0.792 | DB = 0.291
K = 5 | WCSS = 512.4 | Silhouette = 0.689 | DB = 0.574
K = 6 | WCSS = 460.3 | Silhouette = 0.551 | DB = 0.842
K = 7 | WCSS = 408.9 | Silhouette = 0.43 | DB = 1.011
K = 8 | WCSS = 360.8 | Silhouette = 0.337 | DB = 1.133
K = 9 | WCSS = 325.4 | Silhouette = 0.35 | DB = 1.039
K = 10 | WCSS = 291.6 | Silhouette = 0.362 | DB = 0.961
```

```
Best K by silhouette: 4
```

— OR —

(2) **Write a program to build a Knowledge Base using logical rules and facts. [20 marks]**

PROGRAM

```
facts = {('parent', 'Tom', 'Bob'), ('parent', 'Tom', 'Liz'), ('parent', 'Bob', 'Ann'),
         ('male', 'Tom'), ('male', 'Bob'), ('female', 'Liz'), ('female', 'Ann')}
people = {x for f in facts for x in f[1:]}

def parent(x, y): return ('parent', x, y) in facts

def father(x, y): return parent(x, y) and ('male', x) in facts
def mother(x, y): return parent(x, y) and ('female', x) in facts
def grandparent(x, z): return any(parent(x, y) and parent(y, z) for y in people)
def sibling(x, y): return x != y and any(parent(p, x) and parent(p, y) for p in people)

print('father(Tom, Bob):', father('Tom', 'Bob'))
print('grandparent(Tom, Ann):', grandparent('Tom', 'Ann'))
print('sibling(Bob, Liz):', sibling('Bob', 'Liz'))
print('All grandparent pairs:', [(x, y) for x in people for y in people if grandparent(x, y)])

facts.add(('parent', 'Ann', 'Joe'))
people.add('Joe')
print('grandparent(Bob, Joe):', grandparent('Bob', 'Joe'))
```

OUTPUT

```
father(Tom, Bob): True
grandparent(Tom, Ann): True
sibling(Bob, Liz): True
All grandparent pairs: [('Tom', 'Ann')]
grandparent(Bob, Joe): True
```

SLIP NO. 16

AI & ML Practical | 35 Marks

(1) Write a program to implement Depth Limited Search (DLS). [10 marks]

PROGRAM

```
graph = {1: [2, 3], 2: [4, 5], 3: [6], 4: [], 5: [], 6: []}

def dls(node, goal, limit, path=None):
    path = (path or []) + [node]
    if node == goal:
        return path
    if limit <= 0:
        return None
    for child in graph[node]:
        result = dls(child, goal, limit - 1, path)
        if result:
            return result
    return None

print('Depth limit 2:', dls(1, 6, 2))
print('Depth limit 1:', dls(1, 6, 1))
```

OUTPUT

```
Depth limit 2: [1, 3, 6]
Depth limit 1: None
```

(2) Write a program to implement the Backward Chaining inference mechanism. [20 marks]

PROGRAM

```
rules = {'frog': [['croaks', 'eats_flies']], 'green': [['frog']],
         'canary': [['chirps', 'sings']], 'yellow': [['canary']]}
facts = {'croaks', 'eats_flies'}

def backward(goal, depth=0):
    pad = '  ' * depth
    print(pad + 'Proving:', goal)
    if goal in facts:
        print(pad + goal, 'is a known fact')
        return True
    for conditions in rules.get(goal, []):
        if all(backward(c, depth + 1) for c in conditions):
            facts.add(goal)
            return True
    return False

print('Is the animal green?', backward('green'))
print('Is the animal yellow?', backward('yellow'))
```

OUTPUT

```
Proving: green
  Proving: frog
    Proving: croaks
    croaks is a known fact
    Proving: eats_flies
    eats_flies is a known fact
Is the animal green? True
Proving: yellow
  Proving: canary
```

```
    Proving: chirps
Is the animal yellow? False
```

— OR —

(2) **Write a program to implement the K-Means Clustering Algorithm for grouping data into clusters. Use the data set provided in country_data.csv (columns: country, child_mortality, exports, health, imports, income, inflation, life_expectancy, total_fer, gdp). Normalize the data set for KMeans clustering (hint: use StandardScaler()). Apply KMeans with number of clusters = 3. Use different values for k from 1 to 10, use Elbow method to find the ideal value of k. [20 marks]**

PROGRAM

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df = pd.read_csv('country_data.csv')
X = df.select_dtypes(include='number')
X_scaled = StandardScaler().fit_transform(X)

wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_scaled)
    wcss.append(km.inertia_)

plt.plot(range(1, 11), wcss, 'o-')
plt.title('Elbow method')
plt.xlabel('k')
plt.ylabel('WCSS (inertia)')
plt.show()

km = KMeans(n_clusters=3, n_init=10, random_state=42)
df['cluster'] = km.fit_predict(X_scaled)
print(df.iloc[:, [0, -1]].head(10))
print(df['cluster'].value_counts())
```

OUTPUT

Depends on the country_data.csv file given in the exam (elbow plot and cluster labels).

SLIP NO. 17

AI & ML Practical | 35 Marks

(1) **Write a program to implement the Best First Search Algorithm using heuristic values to find a path from a start node to a goal node. [10 marks]**

PROGRAM

```
import heapq

graph = {'A': ['B', 'C'], 'B': ['A', 'D', 'E'], 'C': ['A', 'F'],
         'D': ['B', 'F'], 'E': ['B', 'F'], 'F': ['C', 'D', 'E']}
h = {'A': 6, 'B': 5, 'C': 3, 'D': 2, 'E': 1, 'F': 0}

def best_first(graph, h, start, goal):
    pq = [(h[start], start, [start])]
    visited = set()
    while pq:
        _, node, path = heapq.heappop(pq)
        if node == goal:
            return path
        if node in visited:
            continue
        visited.add(node)
        for nb in graph[node]:
            if nb not in visited:
                heapq.heappush(pq, (h[nb], nb, path + [nb]))
    return None

print('Best First Search path:', ' -> '.join(best_first(graph, h, 'A', 'F')))
```

OUTPUT

```
Best First Search path: A -> C -> F
```

(2) **Write a program to implement Propositional Logic and evaluate logical expressions using operators such as AND, OR, and IMPLIES. [20 marks]**

PROGRAM

```
import itertools

AND = lambda p, q: p and q
OR = lambda p, q: p or q
NOT = lambda p: not p
IMPLIES = lambda p, q: (not p) or q

def evaluate(expr, **values):
    env = {'AND': AND, 'OR': OR, 'NOT': NOT, 'IMPLIES': IMPLIES}
    env.update(values)
    return eval(expr, {'__builtins__': {}}, env)

def truth_table(expr, names):
    print(*names, '|', expr)
    for vals in itertools.product([True, False], repeat=len(names)):
        r = evaluate(expr, **dict(zip(names, vals)))
        print(*[int(v) for v in vals], '|', int(r))

print(evaluate('AND(p, q)', p=True, q=False))
print(evaluate('OR(p, q)', p=True, q=False))
print(evaluate('IMPLIES(p, q)', p=True, q=False))
print()
truth_table('IMPLIES(p, q)', ['p', 'q'])
```

```
print()
truth_table('IMPLIES(AND(p, q), OR(p, q))', ['p', 'q'])
```

OUTPUT

```
False
True
False

p q | IMPLIES(p, q)
1 1 | 1
1 0 | 0
0 1 | 1
0 0 | 1

p q | IMPLIES(AND(p, q), OR(p, q))
1 1 | 1
1 0 | 1
0 1 | 1
0 0 | 1
```

— OR —

(2) **Write a program to calculate and demonstrate the Euclidean Distance between two data points in Machine Learning. The program should accept two input vectors from the user and display the calculated Euclidean Distance. Input: Data Point: 2 3 4, Data Point: 5 7 6. [20 marks]**

Note: (5-2)^2 + (7-3)^2 + (6-4)^2 = 29, so the distance is 5.385. The 5.099 printed in the paper does not match its own inputs.

PROGRAM

```
import math

a = list(map(float, input('Data Point: ').split()))
b = list(map(float, input('Data Point: ').split()))

if len(a) != len(b):
    print('Both vectors must have the same length')
else:
    d = math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))
    fmt = lambda v: [int(x) if x.is_integer() else x for x in v]
    print('Data Point :', fmt(a))
    print('Data Point :', fmt(b))
    print('Euclidean Distance =', round(d, 3))
```

OUTPUT

```
Data Point: 2 3 4
Data Point: 5 7 6
Data Point : [2, 3, 4]
Data Point : [5, 7, 6]
Euclidean Distance = 5.385
```

SLIP NO. 21

AI & ML Practical | 35 Marks

(1) Write a program to implement Iterative Deepening Search (IDS). [10 marks]

PROGRAM

```
graph = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E': [], 'F': []}

def dls(node, goal, limit, path):
    path = path + [node]
    if node == goal:
        return path
    if limit == 0:
        return None
    for child in graph[node]:
        result = dls(child, goal, limit - 1, path)
        if result:
            return result
    return None

def ids(start, goal, max_depth):
    for depth in range(max_depth + 1):
        print('Trying depth limit', depth)
        result = dls(start, goal, depth, [])
        if result:
            return result, depth
    return None, None

path, depth = ids('A', 'F', 5)
print('Path:', ' -> '.join(path), '| found at depth', depth)
```

OUTPUT

```
Trying depth limit 0
Trying depth limit 1
Trying depth limit 2
Path: A -> C -> F | found at depth 2
```

(2) Write a program to implement the Gaussian Naive Bayes Classifier for classifying numerical data. [20 marks]

PROGRAM

```
from sklearn.datasets import make_blobs
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

X, y = make_blobs(n_samples=200, centers=3, n_features=2,
                  cluster_std=1.5, random_state=42)
model = GaussianNB()
model.fit(X, y)

X_test = [[-2, 5], [0, 0], [6, -0.3]]
pred = model.predict(X_test)
print('Prediction:', pred)
print('Accuracy:', accuracy_score([0, 1, 1], pred))
```

OUTPUT

```
Prediction: [0 1 1]
Accuracy: 1.0
```

— OR —

(2) **Write a program to solve the Map Colouring Problem using the Constraint Satisfaction Problem (CSP) approach. [20 marks]**

Note: A, B, C and E are all connected to each other, so 4 colours are needed.

PROGRAM

```
def map_coloring(graph, colors):
    assign = {}
    vertices = list(graph)

    def is_safe(v, c):
        return all(assign.get(n) != c for n in graph[v])

    def solve(i):
        if i == len(vertices):
            return True
        v = vertices[i]
        for c in colors:
            if is_safe(v, c):
                assign[v] = c
                if solve(i + 1):
                    return True
                del assign[v]
        return False

    return assign if solve(0) else None

graph = {'A': ['B', 'C', 'E'], 'B': ['A', 'C', 'D', 'E'], 'C': ['A', 'B', 'E', 'F'],
         'D': ['B', 'E'], 'E': ['A', 'B', 'C', 'D', 'F'], 'F': ['C', 'E']}
colors = ['Red', 'Green', 'Blue', 'Yellow']
result = map_coloring(graph, colors)
if result:
    print('Map Coloring Solution:')
    for v, c in result.items():
        print(v, '->', c)
else:
    print('No solution with', len(colors), 'colors')
```

OUTPUT

```
Map Coloring Solution:
A -> Red
B -> Green
C -> Blue
D -> Red
E -> Yellow
F -> Red
```

SLIP NO. 22

AI & ML Practical | 35 Marks

(1) Write a program to solve the Tower of Hanoi Problem using state-space representation. [10 marks]

PROGRAM

```
from collections import deque

def hanoi(n):
    start = (tuple(range(n, 0, -1)), (), ())
    goal = ((), (), tuple(range(n, 0, -1)))
    queue = deque([(start, [])])
    seen = {start}
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path
        for i in range(3):
            if not state[i]:
                continue
            disk = state[i][-1]
            for j in range(3):
                if i != j and (not state[j] or state[j][-1] > disk):
                    s = list(state)
                    s[i] = state[i][:-1]
                    s[j] = state[j] + (disk,)
                    s = tuple(s)
                    if s not in seen:
                        seen.add(s)
                        queue.append((s, path + ['Move disk ' + str(disk) + ' from ' + 'ABC'[i] + ' to ' + 'ABC'[j]]))

moves = hanoi(3)
for i, m in enumerate(moves, 1):
    print(i, m)
print('Total moves:', len(moves))
```

OUTPUT

```
1 Move disk 1 from A to C
2 Move disk 2 from A to B
3 Move disk 1 from C to B
4 Move disk 3 from A to C
5 Move disk 1 from B to A
6 Move disk 2 from B to C
7 Move disk 1 from A to C
Total moves: 7
```

(2) Write a program to solve the N-Queens Problem using the Backtracking Algorithm. [20 marks]

PROGRAM

```
def solve_queens(n):
    solutions, cols = [], []

    def safe(r, c):
        return all(c != cc and abs(c - cc) != r - rr for rr, cc in enumerate(cols))

    def place(r):
        if r == n:
            solutions.append(cols[:])
            return
        for c in range(n):
            if safe(r, c):
                cols.append(c)
                place(r + 1)
                cols.pop()
```

```
    place(0)
    return solutions

n = int(input('Enter N: '))
sols = solve_queens(n)
print('Number of solutions:', len(sols))
if sols:
    print('First solution:')
    for c in sols[0]:
        print(' '.join('Q' if i == c else '.' for i in range(n)))
```

OUTPUT

```
Enter N: 4
Number of solutions: 2
First solution:
. Q . .
. . . Q
Q . . .
. . Q .
```

— OR —

(2) **Write a program to design a Rule-Based Expert System for simple decision-making. [20 marks]**

PROGRAM

```
rules = [
    ({'fever', 'cough', 'breathless'}, 'Possible chest infection - see a doctor immediately'),
    ({'fever', 'cough'}, 'Probably flu - take rest and fluids'),
    ({'sneezing', 'runny_nose'}, 'Common cold - warm drinks and rest'),
    ({'fever', 'rash'}, 'Possible viral rash - consult a doctor'),
    ({'headache'}, 'Mild headache - rest and drink water'),
]

print('Symptoms: fever, cough, breathless, sneezing, runny_nose, rash, headache')
facts = set(input('Enter your symptoms (comma separated): ').replace(' ', '').split(','))

for cond, advice in rules:
    if cond <= facts:
        print('Diagnosis:', advice)
        break
else:
    print('No rule matched - please consult a doctor')
```

OUTPUT

```
Symptoms: fever, cough, breathless, sneezing, runny_nose, rash, headache
Enter your symptoms (comma separated): fever,cough
Diagnosis: Probably flu - take rest and fluids
```

SLIP NO. 23

AI & ML Practical | 35 Marks

(1) **Write a program to demonstrate the Hill Climbing Algorithm for finding the maximum value of a simple objective function. F(x) = -x^2 + 4x. [10 marks]**

PROGRAM

```
import random

def f(x):
    return -x ** 2 + 4 * x

def hill_climb(f, start, step=0.5, iters=1000):
    x = start
    for _ in range(iters):
        best = max([x + step, x - step], key=f)
        if f(best) <= f(x):
            break
        x = best
    return x, f(x)

random.seed(1)
start = random.randint(-10, 10)
x, val = hill_climb(f, start)
print('Starting point:', start)
print('Maximum found at x =', round(x, 2))
print('Maximum value F(x) =', round(val, 2))
```

OUTPUT

```
Starting point: -6
Maximum found at x = 2.0
Maximum value F(x) = 4.0
```

(2) **Write a program to solve the Map Colouring Problem using the Constraint Satisfaction Problem (CSP) approach. [20 marks]**

PROGRAM

```
def map_coloring(graph, colors):
    assign = {}
    vertices = list(graph)

    def is_safe(v, c):
        return all(assign.get(n) != c for n in graph[v])

    def solve(i):
        if i == len(vertices):
            return True
        v = vertices[i]
        for c in colors:
            if is_safe(v, c):
                assign[v] = c
                if solve(i + 1):
                    return True
                del assign[v]
        return False

    return assign if solve(0) else None

graph = {'A': ['B', 'C'], 'B': ['A', 'C', 'D'], 'C': ['A', 'B', 'E'],
         'D': ['B', 'E'], 'E': ['C', 'D']}
```

```
colors = ['Red', 'Green', 'Blue']
result = map_coloring(graph, colors)
if result:
    print('Map Coloring Solution:')
    for v, c in result.items():
        print(v, '->', c)
else:
    print('No solution with', len(colors), 'colors')
```

OUTPUT

```
Map Coloring Solution:
A -> Red
B -> Green
C -> Blue
D -> Red
E -> Green
```

— OR —

(2) **Write a program or report to compare Supervised Learning and Unsupervised Learning techniques. [20 marks]**

PROGRAM

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score, silhouette_score, adjusted_rand_score

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
knn = KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)
print('Supervised (KNN) accuracy:', round(accuracy_score(y_test, knn.predict(X_test)), 3))

km = KMeans(n_clusters=3, n_init=10, random_state=1).fit(X)
print('Unsupervised (KMeans) silhouette:', round(silhouette_score(X, km.labels_), 3))
print('Agreement with true labels (ARI):', round(adjusted_rand_score(y, km.labels_), 3))

print()
print('Supervised   : labelled data, predicts a label/value (KNN, SVM, Decision Tree)')
print('Unsupervised : unlabelled data, finds groups/structure (K-Means, PCA)')
```

OUTPUT

```
Supervised (KNN) accuracy: 0.978
Unsupervised (KMeans) silhouette: 0.553
Agreement with true labels (ARI): 0.73

Supervised   : labelled data, predicts a label/value (KNN, SVM, Decision Tree)
Unsupervised : unlabelled data, finds groups/structure (K-Means, PCA)
```

SLIP NO. 25

AI & ML Practical | 35 Marks

(1) Write the program to implement the AO* Algorithm. [10 marks]

Note: heuristic values of the leaf nodes are not given, so F, G, H, I, J = 1 are assumed.

PROGRAM

```
graph = {
    'A': [[('B', 2)], [('C', 3)]],
    'B': [[('D', 1), ('E', 4)]],
    'C': [[('F', 2)], [('G', 5)]],
    'D': [[('H', 2)], [('I', 3)]],
    'E': [[('J', 1)]],
    'F': [], 'G': [], 'H': [], 'I': [], 'J': [],
}
h = {'F': 1, 'G': 1, 'H': 1, 'I': 1, 'J': 1}
best = {}

def ao_star(node):
    if not graph[node]:
        return h[node]
    costs = []
    for option in graph[node]:
        costs.append(sum(c + ao_star(child) for child, c in option))
    i = costs.index(min(costs))
    best[node] = graph[node][i]
    return costs[i]

def solution(node):
    print(node, end=' ')
    for child, c in best.get(node, []):
        solution(child)

print('Minimum cost to solve A:', ao_star('A'))
print('Solution graph:', end=' ')
solution('A')
print()
```

OUTPUT

```
Minimum cost to solve A: 6
Solution graph: A C F
```

(2) Write a program to implement the Minimax Algorithm for a two-player game. [20 marks]

PROGRAM

```
tree = [[3, 5], [2, 9]]

def minimax(node, maximizing):
    if isinstance(node, int):
        return node
    values = [minimax(child, not maximizing) for child in node]
    return max(values) if maximizing else min(values)

print('Value of B =', minimax(tree[0], False))
print('Value of C =', minimax(tree[1], False))
print('Value of A (root) =', minimax(tree, True))
```

OUTPUT

```
Value of B = 3
Value of C = 2
Value of A (root) = 3
```

— OR —

(2) **Write a program to implement the Forward Chaining inference mechanism. [20 marks]**

PROGRAM

```
rules = [({'croaks', 'eats_flies'}, 'frog'),
         ({'chirps', 'sings'}, 'canary'),
         ({'frog'}, 'green'),
         ({'canary'}, 'yellow')]
facts = {'croaks', 'eats_flies'}
goal = 'green'

def forward_chain(rules, facts, goal):
    facts = set(facts)
    changed = True
    while changed:
        changed = False
        for cond, result in rules:
            if cond <= facts and result not in facts:
                facts.add(result)
                print(sorted(cond), '=>', result)
                changed = True
                if result == goal:
                    return True
    return goal in facts

print('Goal', goal, 'proved:', forward_chain(rules, facts, goal))
```

OUTPUT

```
['croaks', 'eats_flies'] => frog
['frog'] => green
Goal green proved: True
```